Hello, Fολang!

A Friendly Introduction to Programming with Fολang

Kameswara Rao Mithipati

Preface

Welcome to *Hello, Fολang!*

Learning a programming language should begin with the satisfaction of making something work. It should not begin with every feature the language offers, every architectural choice a large application may require, or every rule contained in its specification.

Hello, Fολang! therefore begins with one file and one clear goal: help you write, build, and run useful Fολang programs without first learning the complete language.

The file is:

```
src/appl.fol
```

It is Fολang's fixed application entry file. It needs no entry-point annotation and no ceremonial `main` function. In a single-source application, this file contains the complete application program.

The promise of this book

By the end of *Hello, Fολang!*, you will be able to:

- create a single-source Fολang application;
- print values and call built-in package methods;
- work with literals and built-in types;
- declare, initialize, infer, and update variables;
- combine values with operators and expressions;
- make decisions with Fολang conditions;
- repeat work with loops;
- traverse arrays and built-in collections;
- use the built-in `co.*` packages;
- add an available third-party library to the project;
- import or alias its published API;
- call that library from `src/appl.fol`; and

- build and run the resulting application.

This is a complete learning goal. When you finish the book, you will have working programs and a useful understanding of Foλang's single-source application model.

It is not the whole language, and it is not intended to be.

One source file, with useful dependencies

The projects in this book use the following source layout:

```
<project>/
└── src/
    └── appl.fol
```

There are no user-created package directories beneath `src/`, and the book does not ask you to create packages or components. Everything you write belongs to the application entry file.

A single-source application does not have to be dependency-free. Foλang's built-in `co.*` packages are available to the entry file, and an application may also use a third-party library. The application source remains one file even when the project includes an external library artifact and `src/appl.fol` imports its API.

This distinction is important:

> Single-source means that the application's own program is contained in `src/appl.fol`. It does not mean that the application cannot use libraries.

The book teaches you how to consume those APIs. Creating your own multi-file package or component belongs to a later stage of learning.

Learning through complete programs

The examples are small enough to understand in one sitting, but each one is a complete application rather than an isolated syntax fragment. You will see a result, change the program, build it again, and observe how that change affects its behavior.

The chapters progress gradually:

1. create and run the first application;
2. print values with built-in package methods;
3. introduce literals, types, and variables;
4. build expressions from values and operators;
5. make decisions with conditions;
6. repeat operations with loops;
7. process arrays and built-in collections;
8. combine the constructs in a small application;
9. add and import a third-party library; and
10. finish with a useful single-source program that builds and runs as one application.

Only the ideas needed for these programs are introduced. A construct is explained when it solves a visible problem, not merely because Foλang contains it.

How to use this book

Type the examples when you can. Run them before changing them. Then make one change at a time and observe the compiler or the program.

When an example uses a built-in package path such as `co.out`, read the complete path before reaching for an alias. When an example introduces an alias or third-party import, notice what becomes shorter and what remains unchanged. When a condition or loop looks unfamiliar, follow the program's behavior rather than translating it immediately into another language's syntax.

The examples use Foλang's own vocabulary. Learning that vocabulary directly will make the programs easier to read and the compiler's feedback easier to understand.

This book and the Language Reference

Hello, Foλang! is an explanatory and practical guide. It selects the rules needed for its programs and presents them in a learning order. It does not define the language.

The **Foλang Language Reference** is the normative definition of Foλang. If this book and the applicable Language Reference differ, the Language Reference governs. Readers who want the exact grammar, complete semantic rules, or behavior outside this book's scope should consult the reference.

The examples in this edition are written for the applicable current language reference and single-source application rules. Toolchain commands may evolve separately, so the edition should be used with its identified compiler and reference version.

Hello, Foλang! — Single-Source Applications

These examples use FoLang's fixed single-source application entry file:

```
<project>/
└── src/
    └── appl.fol
```

Each example is a complete application unless it is explicitly labeled as a third-party library template. Use one example at a time as `src/appl.fol`. A single-source project has no user-created package directories beneath `src/`.

The examples follow one simple rhythm:

1. Read the developer's question.
2. Predict what the program will do.
3. Run the program.
4. Compare the result with the expected output.
5. Make the suggested change and run it again.

> **Reference note:** This is a teaching guide. The **FoLang Language Reference** is the normative definition of the language. If this guide and the applicable Language Reference differ, the Language Reference governs.

1. Can I print something?

> **Developer:** Do I need to create a `main` function before I can print a message?

> **Guide:** No. `src/appl.fol` is already the application entry file.

```
co.out.println("Hello, FoLang!");
```

Expected output:

```
Hello, FoLang!
```

The entry file needs no entry-point annotation and no ceremonial `main` function.

Try it: Change the message and run the application again.

2. How do I create variables and make a choice?

> **Developer:** Do I have to write the type of every variable?
>
> **Guide:** No. You may declare a type explicitly, or use `:=` and let FoLang infer it from the initializer.

```
name co.lang.string = "Kameswara";
score := 82;

co.out.print("Hello, ");
co.out.println(name);

(score >= 50).then({
    co.out.println("Result: pass");
}).default({
    co.out.println("Result: try again");
});
```

Pause and predict the output before continuing.

Expected output:

```
Hello, Kameswara
Result: pass
```

The first binding has an explicit `co.lang.string` type. The second obtains its type from `82`.

`then` handles the true branch. A final branch without another condition is written with `default`. When another condition is needed, FoLang uses `otherwise(condition).then(...)`.

Try it: Change `score` to `40` and run the application again.

3. How do I repeat work?

> **Developer:** How can I add the numbers from one through five?
>
> **Guide:** Use a condition with `loop`, updating the values on every iteration.

```
limit := 5;
current := 1;
total := 0;

(current <= limit).loop({
    total += current;
    current += 1;
});

co.out.print("Total: ");
co.out.println(total);
```

Expected output:

```
Total: 15
```

`loop` has one condition and one body. When the condition becomes false, repetition ends.

Try it: Change `limit` to `10`. Predict the new total before running the program.

4. How do I visit every array element?

> **Developer:** Can I process every score without writing a separate index operation for each one?
>
> **Guide:** Yes. An array exposes `each` for element-by-element iteration.

```
scores co.lang.int->([5]) = [78, 92, 66, 85, 73];
total := 0;

scores.each(_, score, {
    co.out.println(score);
    total += score;
});

co.out.print("Total: ");
co.out.println(total);
```

Expected output:

```
78
92
66
85
73
Total: 394
```

The first `each` binding is the index. This application does not need it, so `_` discards it. The second binding receives the element.

`each` performs the iteration itself. Do not append `.loop(...)` to it.

Try it: Replace `_` with `index`, then print `index` before each score.

5. Can I find a number in an array?

> **Developer:** I have an array. How do I ask whether it contains a particular value?
>
> **Guide:** Use `contains` and make a choice from its Boolean result.

```
number co.lang.int = 30;
numbers co.lang.int->([5]) = [10, 20, 30, 40, 50];

numbers.contains(number).then({
    co.out.print("Number ");
    co.out.print(number);
    co.out.println(" is present.");
}).default({
    co.out.print("Number ");
```

```
    co.out.print(number);
    co.out.println(" is not present.");
});
```

Expected output:

```
Number 30 is present.
```

Try it: Change `number` to `100`. The array does not contain that value, so the `default` branch should run.

6. Can I solve the same problem with a List?

> **Developer:** Does changing from an array to a List require a completely different search?
>
> **Guide:** No. The standard List API also exposes `contains`.

```
number co.lang.int = 30;

numbers co.core.List->(co.lang.int) =
    co.core.List[10, 20, 30, 40, 50];

numbers.contains(number).then({
    co.out.print("Number ");
    co.out.print(number);
    co.out.println(" is present.");
}).default({
    co.out.print("Number ");
    co.out.print(number);
    co.out.println(" is not present.");
});
```

Expected output:

```
Number 30 is present.
```

The array and the List are different data structures, but they provide a consistent containment operation. Consistent operations do not make the data structures identical; their storage, access, and mutation behavior may differ.

Try it: Change `number` to `100` and confirm that the other branch runs.

7. How do Map keys and values differ?

> **Developer:** A Map contains keys and values. What does `contains` search?
>
> **Guide:** `contains` searches keys. Use `containsVal` when you want to search values.

```
number := 100;

numbers co.core.Map->(
    key=co.lang.int,
    val=co.lang.int
) = co.core.Map{
    10: 10,
    100: 20,
    50: 30
};

numbers.contains(number).then({
    co.out.print(number);
    co.out.println(" is a key.");
}).default({
    co.out.print(number);
    co.out.println(" is not a key.");
});

numbers.containsVal(number).then({
    co.out.print(number);
    co.out.println(" is a value.");
}).default({
    co.out.print(number);
    co.out.println(" is not a value.");
});
```

Pause and predict both results.

Expected output:

```
100 is a key.
100 is not a value.
```

The Map has a key `100`, but its values are `10`, `20`, and `30`.

Try it: Change `number` to `20`. Which containment operation succeeds now?

8. Can a Set use the same containment question?

> **Developer:** What if I need a collection of unique values?
>
> **Guide:** Use a Set. Its constructor uses parentheses, and its `contains` operation asks whether an element is present.

```
number := 100;

numbers co.core.Set->(co.lang.int) =
    co.core.Set(10, 100, 50);

numbers.contains(number).then({
    co.out.print("Number ");
    co.out.print(number);
    co.out.println(" is present.");
}).default({
    co.out.print("Number ");
    co.out.print(number);
    co.out.println(" is not present.");
});
```

Expected output:

```
Number 100 is present.
```

Arrays, lists, maps, and sets may share an operation name such as `contains`, but each collection retains its own meaning and behavior.

Try it: Add `100` to the Set constructor a second time and observe how the installed standard-library implementation presents the resulting Set.

9. Can I iterate over a Range?

> **Developer:** Do I have to construct a collection just to work with a sequence of numbers?

> **Guide:** No. A finite Range can describe the sequence directly.

```
numbers := 1 .. 5;

numbers.each(_, value, {
    co.out.println(value);
});

numbers.contains(3).then({
    co.out.println("The range contains 3.");
}).default({
    co.out.println("The range does not contain 3.");
});
```

Expected output:

```
1
2
3
4
5
The range contains 3.
```

`1 .. 5` includes both endpoints. FoLang supports other boundary combinations, including half-open ranges such as `0 ..< 5`. Consult the Language Reference when you need the complete range syntax.

Try it: Change the declaration to `numbers := 0 ..< 5;` and predict the values that will be printed.

10. How do I use a third-party library?

> **Developer:** Can my single-source application use an external library?
>
> **Guide:** Yes. Single-source means that your own application source remains in `src/appl.fol`; it does not mean that the application must be dependency-free.

Place the prebuilt `.folenc` artifact supplied by the library publisher in the project-root `lib/` directory:

```
<project>/
├── lib/
│   └── example.folenc
└── src/
    └── appl.fol
```

The publisher's documentation tells you which import form and public names the artifact exposes.

Projected library surface

A projected library is imported with `library=`:

```
@co.ddap.import(library="example", as="ext")

ext.greet("FoLang");
```

Packaged/open package surface

A packaged artifact exposes selected package contexts. Import one with `package=`:

```
@co.ddap.import(package="example.text", as="text")

text.greet("FoLang");
```

These are templates: replace `example`, the package path, alias, and `greet` call with the exact names documented by the library publisher.

Important rules:

- Put `@co.ddap.import` in the file preamble, before executable statements.
- Use `as=` to create the file-local import alias.
- Do not write a semicolon after the directive.
- The alias must be a valid, unambiguous identifier in that file.
- Call the imported API through the alias, such as `ext.greet(...)`.
- You are consuming a prebuilt dependency; you are not creating a package or component.

Build and run the project with the commands documented for your installed `folcc` release. The Language Reference defines the project and source semantics, while the applicable toolchain documentation supplies the exact command-line flags.

11. Why does `co.*` need no import?

> **Developer:** Third-party APIs need an import. Why can I call `co.out.println` directly?
>
> **Guide:** The `co` root belongs to FoLang's installed standard library and is made available automatically.

The compiler loads the installed standard-library artifact before it parses the application source. That is why this is valid without an import:

```
co.out.println("Hello from the standard library.");
```

You may use the standard APIs exported by the installed standard-library version. Their exact declarations and signatures remain governed by the applicable Language Reference and standard-library artifact.

You may shorten a built-in `co.*` path with a file-local alias:

```
@co.ddap.alias(co.out, as="out")

out.println("Hello through an alias.");
```

`@co.ddap.alias` does not remove the complete path; `co.out.println` remains valid in the same file.

12. Can I create my own data types?

> **Developer:** So far I have used built-in types and APIs. Can FoLang represent types from my own problem?
>
> **Guide:** Yes, but that is deliberately beyond the promise of this book.

Hello, Fολang! is complete within its single-source learning boundary. It teaches you to create, understand, build, and run useful programs from `src/appl.fol`, including programs that consume third-party libraries. It does not introduce the larger application structures used to model and organize your own domain.

That question begins the next stage of the journey.

> If you enjoyed this journey and would like to learn how FoLang approaches and structures a larger application step by step, please have a look at **Think Like FoLang**.